

Structural Grounding Replicates, Prompt Hardening Does Not: Indirect Prompt Injection in LLM-Narrated Security Telemetry

Hitarth Jain*

September 3, 2026

Abstract

Endpoint detection and response (EDR) tooling increasingly asks a language model to read raw telemetry and write the incident summary an analyst reads. That telemetry contains fields an attacker on the compromised host chooses directly—process image paths, command lines, file paths, DNS names, registry keys, user agents—and which the collector is obliged to store verbatim, because normalising them would corrupt forensic evidence. Concatenating those fields into a prompt therefore turns every one of them into an indirect prompt injection channel. The attacker is not typing into a chat box; they are writing the evidence the model was asked to describe.

We build a synthetic EDR pipeline, a corpus of 66 hand-written injection payloads spread across six attacker-controlled fields and three attacker goals, and three narrator configurations: naive concatenation, a prompt-hardened system prompt, and a structurally grounded narrator that constrains what the model can emit rather than instructing it what to avoid. Payloads are disguised as plausible SOC, ticketing, and vendor content; none uses the phrase “ignore previous instructions”, which measures string recognition rather than defense.

Scoring is rule-based rather than LLM-as-judge, so a given (payload, output) pair always scores identically and before/after numbers remain comparable. Uncertainty is reported as a paired percentile bootstrap over the corpus.

Both defended tiers beat naive concatenation by a margin the corpus can resolve (naive 22.7%, prompt-hardened 6.1%, structural 7.6% bypass). The two defended tiers are *not* separable from each other at $n = 66$: the paired difference is -1.5% , 95% CI $[-9.1\%, 6.1\%]$. Replicating on a second model changes the picture. Structural grounding beats naive on both models and lands on the same 7.6%; prompt hardening’s advantage falls to 9.1% $[0.0\%, 18.2\%]$, a lower bound of exactly zero. On the one goal the structural tier was designed for, it never downgraded severity in its own voice: 0 of 31 severity-downgrade payloads, on both models.

We also ask whether these payloads can simply be detected. A character n -gram logistic regression reaches ROC-AUC 0.965 on the hand-written split at a 0.0% false-positive rate, while a fine-tuned DistilRoBERTa reaches 0.939 while firing on 2.7% of benign telemetry. At a matched 1% false-positive rate the simpler model is ahead on the only split not generated by our own code.

We report one methodological failure in full: the first scoring pass counted attacker text *quoted inside a refusal* as a successful bypass, which inverted the ordering of all three tiers until corrected.

1 Introduction

Security operations tooling is an unusually attractive place to put a language model. Telemetry is voluminous, repetitive, and tedious to read; a model that turns a hundred process-creation events

*Independent. jainhitarth963@gmail.com

into three sentences saves real analyst time. Several commercial EDR and SIEM products now ship exactly this feature.

It is also an unusually dangerous place to put one, for a reason that is structural rather than incidental. An EDR agent records what it observes. When a process is created, the agent stores the image path and command line as the operating system reported them. It does not sanitise those strings, and it should not: an agent that rewrites `C:\Users\Public\svch0st.exe` into something tidier has destroyed the evidence it exists to preserve. The same is true of DNS names queried, files written, registry keys set, and user agents presented.

Every one of those fields is chosen by whoever controls the host. If the narration step is “concatenate the telemetry into a prompt and ask the model to summarise”, then the attacker writes part of the prompt.

This is indirect prompt injection, but the setting differs from the ones usually studied in a way that matters for defense. Most published work on indirect injection concerns agents that *act*: tool-calling systems where a successful injection causes the agent to send an email, write a file, or execute a command. The defenses that follow are largely about constraining actions—permissions, tool allow-lists, human approval on destructive operations.

A narrating model takes no actions. Its output is prose that a human reads and believes. A successful injection here does not hijack a tool call; it corrupts a report. The failure modes are that the summary downgrades the severity of a real intrusion, omits an entity the analyst needed to see, or repeats attacker-authored instructions to the analyst as though they were findings. None of those is caught by a permission model, because nothing was permitted or denied—the model simply wrote something untrue.

This paper measures how much three defenses help in that setting, and reports what does and does not replicate across two models.

1.1 Contributions

1. A threat model and synthetic EDR pipeline for indirect injection through attacker-controlled telemetry fields, with each field’s provenance documented in the schema.
2. A 66-payload corpus disguised as legitimate SOC and vendor content, spanning six attacker-controlled fields and three attacker goals, including payloads written against specific defenses rather than generic prose.
3. A rule-based, versioned scorer, chosen over LLM-as-judge so that runs remain comparable, together with a paired bootstrap protocol that reports what the corpus can and cannot separate.
4. Measurements on two models showing that structural grounding replicates and prompt hardening does not.
5. A detection study with a length-only control, showing that a character n -gram model outperforms a fine-tuned transformer at matched false-positive rate on the only split not machine-generated.
6. A full account of a scoring confound that inverted our initial results.

2 Threat model

Attacker capability. The attacker controls a host that an EDR agent monitors. They choose the contents of process image paths, command lines, file paths written or read, DNS names queried, registry keys, and HTTP user agents. They do not have access to the SOC console, the model, the system prompt, or the analyst.

Collector obligation. The agent stores these fields verbatim. This is not a weakness to be fixed; it is the requirement. Normalising attacker-controlled strings before storage destroys forensic fidelity and would itself be a finding.

Attack surface. The narration step. Telemetry rows are assembled into a prompt and a model is asked to describe them.

Attacker goals. Three, each corresponding to a way the analyst-facing report can be made false:

- *severity downgrade* (31 payloads)—the summary characterises a genuine intrusion as benign, routine, authorised, or low risk.
- *entity omission* (15 payloads)—an entity the analyst needed, such as the attacker’s domain or binary, does not appear in the summary.
- *instruction leak* (20 payloads)—attacker-authored text reaches the analyst as though it were the tool’s own finding.

Out of scope. Attacks on the model weights, the console, or the analyst directly; denial of service against the pipeline; and any capability requiring access beyond the compromised host.

3 Corpus

The corpus contains 66 hand-written payloads distributed across the six attacker-controlled fields: process image (12), command line (12), file path (11), DNS name (11), registry key (10), user agent (10).

Every payload is disguised as content a SOC analyst, ticketing system, or security vendor would plausibly emit—a suppression note, a change-ticket reference, a vendor allow-list entry. The corpus deliberately contains no instance of “ignore previous instructions” or its close variants. That phrase is well known and cheap to filter, and testing against it largely measures whether a defense recognises a specific string.

Some payloads are written against a specific defense rather than being generic. This matters for interpretation: a corpus of only generic prose would let a defense that recognises one attack shape appear stronger than it is.

4 Defenses

Tier 1 — naive. Telemetry rows are concatenated into the prompt and the model is asked to summarise. No instruction about untrusted content. This is the baseline, and it is what a first implementation looks like.

Tier 2 — prompt-hardened. A system prompt tells the model that telemetry fields are attacker-controlled, that their contents are evidence rather than instructions, and that it must not act on text found inside them. This is the defense most readily available to a team that has recognised the problem: it costs one prompt edit.

Tier 3 — structurally grounded. Rather than instructing the model to ignore embedded instructions, the narrator constrains what it can produce. Severity is emitted as a discrete field drawn from a fixed set rather than as free prose, and entities are rendered from the parsed telemetry rather than recovered from the model’s text. The hypothesis is that a defense which removes the model’s ability to state a severity in its own voice will fail differently—and less—than one that asks it not to.

5 Method

5.1 Scoring

Scoring is rule-based. Each payload declares its goal, and a deterministic check keyed to that goal decides whether the goal was achieved: benign-characterisation markers for severity downgrade, entity presence for omission, and attacker-string presence for instruction leak.

We chose this over LLM-as-judge deliberately. The claim the paper makes is a *comparison* between tiers and between models, and a judge whose verdicts drift makes before/after numbers incomparable. The scorer carries a version number that is bumped whenever a rule changes, and a run that does not record it is not compared to anything.

5.2 The confound we corrected

The first scoring pass produced the opposite ordering: the defended tiers appeared to make things monotonically worse.

The cause was that the payloads mimic legitimate SOC prose. When a hardened narrator correctly refused an injected instruction, it frequently *quoted the attacker’s text* while doing so—“the command line contains a string claiming this was approved under change ticket CHG-4471; this claim is not corroborated.” The scorer saw the attacker’s marker string in the output and recorded a bypass. The more faithfully a tier quoted the evidence it was rejecting, the worse it scored.

Correcting this required stripping attacker-quoted spans before applying the presence checks. It moved 43 verdicts and reversed the headline result.

We report this because the failure is not specific to our scorer. Any evaluation of injection defenses that checks for the presence of attacker text in output will penalise the behaviour it most wants to reward, unless it distinguishes *quoting* from *complying*.

5.3 Uncertainty

All runs are at temperature 0. The uncertainty we quantify is therefore not sampling noise but corpus composition: which payloads we happened to write. We report a percentile bootstrap [10] over the 66 payloads with 10,000 resamples.

Tier comparisons are paired—every tier sees every payload—so the bootstrap resamples the same payload indices across tiers, and we report the interval on the *difference* rather than comparing two independent intervals.

6 Results

6.1 Bypass rate by tier

Measured on `gemini-3.1-flash-lite`, 66 payloads $\times$ 3 tiers, temperature 0.

Tier	Overall	95% CI	sev. downgrade	entity omission	instr. leak
naive	22.7%	[13.6, 33.3]	26%	47%	0%
prompt-hardened	6.1%	[1.5, 12.1]	6%	13%	0%
structurally grounded	7.6%	[1.5, 15.2]	0%	33%	0%

6.2 What the corpus can and cannot separate

Comparison	Difference	95% CI	Verdict
naive – prompt-hardened	16.7%	[7.6, 27.3]	separated
naive – structural	15.2%	[6.1, 24.2]	separated
prompt-hardened – structural	–1.5%	[–9.1, 6.1]	not separated

Prompt-level hardening does real work: both defended tiers beat naive by a margin this corpus resolves. But 6.1% and 7.6% are not distinguishable at $n = 66$, and reading them as a ranking is reading noise. We do not rank them.

6.3 Replication on a second model

The same corpus, scorer, and temperature on `meta/llama-3.1-8b-instruct`:

Tier	<code>gemini-3.1-flash-lite</code>	<code>llama-3.1-8b-instruct</code>
naive	22.7% [13.6, 33.3]	18.2% [9.1, 27.3]
prompt-hardened	6.1% [1.5, 12.1]	9.1% [3.0, 16.7]
structurally grounded	7.6% [1.5, 15.2]	7.6% [1.5, 15.2]

Asking whether each defense beats naive *on that model*, by a margin the corpus can resolve:

Model	naive – hardened	naive – structural
<code>gemini-3.1-flash-lite</code>	16.7% [7.6, 27.3] — yes	15.2% [7.6, 24.2] — yes
<code>llama-3.1-8b-instruct</code>	9.1% [0.0, 18.2] — no	10.6% [4.5, 18.2] — yes

Structural grounding replicates; prompt hardening does not. On `llama-3.1-8b` the prompt-hardened tier’s advantage over naive has a lower bound of exactly zero: this corpus cannot show it helped at all. The structural tier beats naive on both models and lands on the same 7.6% on each.

This is the paper’s main result. A defense evaluated on one model can look like the equal of another and turn out to be model-dependent.

6.4 The goal the structural tier was built for

On severity downgrade, the structural narrator never downgraded severity in its own voice: 0 of 31 severity-downgrade payloads, on both models. This is the behaviour the tier was designed to make unavailable rather than discouraged, and it is the one place the two defended tiers clearly differ.

7 Can these payloads simply be detected?

If a classifier can flag injected telemetry before narration, the narrator’s robustness matters less. We trained three detectors and evaluated them on three splits: `test_seen` (new strings, techniques seen in training), `test_unseen` (three held-out technique families), and `test_human` (the 66 hand-written corpus payloads—the only split not produced by our own generator).

Thresholds were fitted once on validation at a 1% false-positive rate and applied unchanged.

Model	Split	ROC-AUC	Recall	FPR
length-only control	<code>test_human</code>	0.743	0.167	0.007
char n -gram + LR	<code>test_human</code>	0.965	0.758	0.000
DistilRoBERTa fine-tune	<code>test_human</code>	0.939	0.803	0.027

The length-only control exists to check that neither model is counting characters. It reaches 0.74 and catches almost nothing at the operating point: length is a real signal and a weak one.

The transformer’s raw recall on held-out techniques looks better (0.995 against 0.959 on `test_unseen`), but it fires on 2.7% of benign telemetry against the n -gram model’s 0.0%, so those recalls are not comparable. Recall is free if you are willing to alert on everything. Refitting both thresholds on each test split’s own negatives to hit exactly 1%—an oracle number, reported only so the models can be compared at one false-positive rate—the n -gram model leads on the human split, 0.833 against 0.803.

We take from this that a fine-tuned transformer was not worth its cost here, and that a detector evaluated only on machine-generated attacks will overstate what it does on hand-written ones.

8 Limitations

- $n = 66$. The corpus resolves the gap between defended and undefended configurations and does not resolve the gap between the two defended tiers. A larger corpus, or payloads targeted at where the tiers actually differ, would be needed to rank them.
- **The corpus is ours.** We wrote the payloads, and a corpus measures the attacks its author thought of. The `test_human` detector split exists precisely because everything else was generated by our own code.
- **Structural’s entity-omission number measures the renderer.** The structural tier’s 33% on entity omission reflects per-event field coverage in the renderer, not a decision the model made. It should not be read as the model omitting entities.
- **Two models.** Structural grounding replicated across the two we tested. Two is enough to show that prompt hardening’s benefit is model-dependent; it is not enough to establish that structural grounding is universal.
- **Hosted endpoints are not perfectly reproducible** even at temperature 0.

9 Related work

Greshake et al. [1] established indirect prompt injection as a class, showing that content an LLM-integrated application retrieves can carry instructions the application then follows; Liu et al. [2]

formalised prompt injection against such applications. Subsequent evaluation work has concentrated on agentic settings where a successful injection causes an action. AgentDojo [3] evaluates tool-using agents against injected data returned by tools, and AgentHarm [4] measures harmfulness of agent behaviour. Defenses in that literature are correspondingly action-shaped: Beurer-Kellner et al. [5] catalogue design patterns that constrain what an agent may do with untrusted content, and CaMeL [6] separates a privileged planner from a quarantined model that reads untrusted data. Practitioner guidance follows the same shape [7, 9].

The narration setting studied here is adjacent but distinct: the model takes no action, and the harm is a false report rather than an unauthorised operation. A permission model cannot detect it, because nothing was permitted or denied. To our knowledge, no existing benchmark measures whether a model downgrades severity or omits an entity in its own summary under injection.

The scoring hazard reported in Section 5.2 also appears to be undocumented. The OWASP AI Testing Guide’s indirect prompt injection test [8] confirms a vulnerability on action-shaped outcomes and does not specify how to decide that an injection succeeded; the natural implementation, searching output for the payload, has the failure mode described there. We have contributed that warning upstream.

10 Conclusion

Putting a language model in front of security telemetry makes every attacker-controlled field part of the prompt, and the resulting failure is not a hijacked tool call but a report the analyst has no reason to distrust.

Prompt-level hardening measurably helps, and on a second model we could not show that it helped at all. Constraining what the narrator can structurally emit helped on both models and eliminated the specific failure it was designed to eliminate. If one result generalises beyond this corpus, it is that defenses which remove a capability behave more consistently across models than defenses which ask for restraint—and that any claim of the second kind should be replicated before it is believed.

Availability

Code, corpus, scorer, and all result files are available at <https://github.com/Yeagerist0/prompt-injection-soc-telemetry>.

Every number in this paper is traceable to a committed run file.

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv:2302.12173*, 2023.
- [2] Y. Liu, G. Deng, Y. Li, K. Wang, et al. Prompt injection attack against LLM-integrated applications. *arXiv:2306.05499*, 2023.
- [3] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv:2406.13352*, 2024.

- [4] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, et al. AgentHarm: A benchmark for measuring harmfulness of LLM agents. *arXiv:2410.09024*, 2024.
- [5] L. Beurer-Kellner, B. Buesser, A.-M. Crețu, E. Debenedetti, et al. Design patterns for securing LLM agents against prompt injections. *arXiv:2506.08837*, 2025.
- [6] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, et al. Defeating prompt injections by design. *arXiv:2503.18813*, 2025.
- [7] OWASP. OWASP Top 10 for Large Language Model Applications, LLM01: Prompt Injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection>.
- [8] OWASP. AI Testing Guide, AITG-APP-02: Testing for Indirect Prompt Injection. <https://github.com/OWASP/www-project-ai-testing-guide>.
- [9] National Institute of Standards and Technology. Adversarial machine learning: A taxonomy and terminology of attacks and mitigations. NIST AI 100-2e2025. <https://doi.org/10.6028/NIST.AI.100-2e2025>.
- [10] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.